Demo Video Script

Narration script & step-by-step live-demo walkthrough

Multi-Agent Incident Response System — Track 2 Capstone
github.com/bgolsen/agents-final-project

Demo Video Script — Multi-Agent Incident Response System

Target length: 9-10 minutes. This is a speaking script with stage directions (`[ACTION: ...]`). Read the narration in your own words — don't recite it verbatim — but hit every bolded beat, since those map directly to rubric line items (noted in `[Rubric: ...]` tags; those tags are for you, cut them from what you say out loud).

Companion docs: `ARCHITECTURE.md` (diagrams, framework justification, tradeoffs) and `README.md` (setup/run instructions).

---

0. Before you hit record

- [ ] `.env` has a working `GOOGLE_API_KEY` and `LANGSMITH_API_KEY` (confirm by running one incident end-to-end beforehand, off-camera, so you know the keys/model are live and you're not debugging on camera).
- [ ] Close any stray terminals bound to ports 8000-8004 (`Get-NetTCPConnection -LocalPort 8000,8001,8002,8003,8004` in PowerShell should return nothing).
- [ ] Terminal font size large enough to read on a recording (14-16pt+).
- [ ] Have these open in tabs, ready to alt-tab to: `ARCHITECTURE.md` (rendered preview, e.g. in VS Code or on GitHub, so the Mermaid diagrams render), a terminal at the repo root with the venv activated, and your LangSmith project page (smith.langchain.com → your `incident-response-adk` project).
- [ ] Quiet room, mic close, no notifications popping up. **[Rubric: production quality — clear audio, legible screen, good pacing]**

---

1. Cold open (0:00–0:30)

[ACTION: face cam or just screen with your terminal at the repo root]

> "Hi, I'm [name]. This is my capstone for Track 2: a multi-agent incident response system. When a production incident fires, four specialist agents — built on Google ADK and talking to each other over the A2A protocol — triage it, diagnose the root cause against a knowledge base of past incidents, propose a remediation plan, and stop for a human to approve it before anything actually

> executes. n8n handles alert intake and routing, and LangSmith traces the whole reasoning chain. Let me walk through the architecture, then show it running live, including what happens when something fails."

---

2. Architecture (0:30–2:45) — [Rubric: presentation clarity, conceptual grounding]

[ACTION: switch to `ARCHITECTURE.md`, scroll to the §4 sequence diagram]

> "Here's the flow. An alert comes in — from n8n, or directly to the coordinator API. The coordinator calls the Monitoring Agent over A2A to triage it. That triage report gets handed to the Diagnostic Agent, which doesn't just guess — it explicitly decomposes 'what's causing this' into 2 to 4 sub-questions, answers each one using tools, and only then ranks root-cause hypotheses with confidence scores and cited evidence. That's the hierarchical planning requirement: it's not hard-coded, the model produces a different decomposition per incident, and it's fully inspectable in the trace."

[ACTION: scroll to the agent role table, §2]

> "Four specialist agents, each a separate process, each its own A2A server — Monitoring, Diagnostic, Remediation, Postmortem — plus a Coordinator that's deliberately *not* another LLM. I made that call on purpose: human-in-the-loop needs a hard stop between 'plan produced' and 'plan executed,' and that's easier to guarantee with explicit code than with an LLM deciding when to hand off control. That's in §7 of the architecture doc if you want the full tradeoff writeup, including the LangGraph alternative I considered and rejected."

[ACTION: scroll to §3, framework table]

> "Four frameworks, two were required: Google ADK for the agents themselves — structured output schemas plus tool use, and built-in error-callback hooks that are what make the graceful-degradation story work. A2A protocol for the actual wire communication between agents — these are real HTTP/JSON-RPC calls between separate processes, not one function calling another. n8n fronts the system for alert intake and severity-based routing. And LangSmith traces every stage of the reasoning chain, tagged per incident."

---

3. Launch the system (2:45–3:15)

[ACTION: terminal at repo root, venv activated]

> "Let's run it. I'll start the four agents and the coordinator."

```
.\scripts\start_agents.ps1
```

[ACTION: 5 new terminal windows pop up, titled "Agent: monitoring (:8001)" etc. — briefly tile or point at them. If your terminal profile doesn't show the title bar text, each window also prints its own name/port on startup — e.g. "Starting A2A server for 'diagnostic' at http://127.0.0.1:8002" — so you can still tell them apart for §5.]

> "Five separate windows, five separate OS processes. Let me prove that — here's the Monitoring Agent's A2A agent card, over plain HTTP."

```
curl http://localhost:8001/.well-known/agent-card.json
```

[ACTION: point out the JSON response — `name`, `skills`, the tool descriptions]

> "That's a real A2A agent card: its name, its declared skills, the tools it exposes. Any A2A-compliant client could talk to this agent, not just my coordinator."

---

4. Demo run #1 — full incident, live reasoning, HITL (3:15–7:00)

[Rubric: demo quality — end-to-end, agent interactions, HITL checkpoint]

[ACTION: new terminal, repo root]

```
python -m incident_response.run_incident --no-spawn --scenario checkout-latency-spike
```

> "This scenario simulates a checkout-service latency and error spike. I'm running it interactively this time — no auto-approve — so you can see the human decision happen live, not scripted."

[ACTION: let the Monitoring Agent's triage output print]

> "That's the Monitoring Agent — it called two tools, got a metrics snapshot and recent logs, flagged three anomalous metrics, and it's a live Gemini call, not canned text."

[ACTION: let the Diagnostic Agent's output print — pause here]

> "This scenario also has a simulated failure baked in: the log API times out once. Watch — [point at the data_gaps / unresolved_uncertainty line] — the agent doesn't crash, it records the gap explicitly, lowers its confidence, and reasons around it. That's the first layer of error handling: the LLM itself adapting to a failed tool call inside its own reasoning, via an `on_tool_error_callback`. It still lands on the right root cause — the payment-gateway dependency timing out — and cites the matching past incident from the knowledge base."

[ACTION: let the Remediation Agent's plan print]

> "The Remediation Agent doesn't propose one action — it builds a goal hierarchy: 'Immediate mitigation' with the fastest safe fix, and 'Preventive follow-up' for the underlying cause, each step with a risk rating and an explicit rollback plan."

[ACTION: the HITL prompt appears — `Approve / Reject / Modify goal? [a/r/m]:`]

> "And now it stops. Nothing has executed yet. This is the human-in-the-loop checkpoint, and I want to show it's not a rubber stamp — instead of just approving the recommended plan, I'm going to pick the *other* goal."

[ACTION: type `m` for modify]

```
Which goal should run instead?: Preventive follow-up
Why are you changing the plan?: Want the root-cause fix now, not just the mitigation
Your name: <your name>
```

> "I just told it to run 'Preventive follow-up' instead of what it recommended."

[ACTION: let execution results print]

> "And there — it executed the *steps from the goal I picked*, not the agent's recommendation. That's a genuine behavior change from a human decision, which is the bar for a meaningful HITL checkpoint,

> not a confirmation dialog."

[ACTION: let the postmortem print]

> "Postmortem Agent closes it out — root cause, impact, and critically, it records exactly what I decided and why, so that decision is auditable."

---

5. Edge case — kill an agent mid-flight (7:00–8:15)

[Rubric: demo quality — handling of at least one failure/edge case]

> "That first run already showed one failure mode — a tool timing out and the agent adapting. I want to show the *other* resilience layer: what happens if an entire agent process goes down, not just one tool call."

[ACTION: switch to the Diagnostic Agent's terminal window (port 8002)]

> "I'm going to close the Diagnostic Agent's window entirely — simulating a crash."

[ACTION: close that terminal / Ctrl+C it]

[ACTION: back in your driver terminal]

```
python -m incident_response.run_incident --no-spawn --scenario auth-cert-expiry --auto-approve
```

> "I'll auto-approve this one so we get straight to the interesting part."

[ACTION: let it run — point at the Diagnostic Agent section]

> "There — the coordinator tried to reach the Diagnostic Agent over A2A, retried once, couldn't connect, and fell back to calling the same deterministic root-cause logic *locally*, in-process, instead of aborting the incident. You can see it flagged right there in the coordinator's resilience notes, and the incident still completes — triage, a (simpler, rule-based) diagnosis, a plan, and a postmortem, all

> the way through. Two independent layers: a tool fails, the LLM adapts; an entire agent fails, the coordinator adapts. Neither one crashes the incident."

[ACTION: restart the diagnostic agent for the rest of the demo, if needed]

```
python -m incident_response.a2a_server diagnostic
```

---

6. Observability — LangSmith & n8n (8:15–9:15)

[Rubric: observability, framework selection]

[ACTION: switch to your browser, LangSmith project page]

> "Every stage you just watched print to the terminal is also a traced span in LangSmith — triage, diagnosis, remediation planning, execution, postmortem — nested under one run per incident, tagged with the incident ID. [click into the most recent trace] Here's the run from the checkout incident — you can see the modify decision, the retry, all of it, fully inspectable after the fact."

[ACTION: switch to n8n, if you have it running, or the workflow JSON in your editor]

> "And this is the n8n side — alert intake, severity-based routing to a notification channel, and a poll loop that waits for the human decision before sending a completion notification. It's decoupled from the reasoning entirely; the coordinator API works identically whether n8n is in front of it or not."

---

7. Wrap-up (9:15–10:00)

[Rubric: tradeoff analysis, limitations]

> "A couple of honest limitations, covered in more depth in the architecture doc: the degraded-mode fallback ranks past incidents by keyword overlap, not real reasoning, so it's noticeably worse than the LLM path when it kicks in — that's intentional, it's a cheap safety net, not a second reasoning engine. And chaos injection here is per-process rather than per-incident, which is fine for a demo but wouldn't hold up in a long-running deployment.

> That's the system — five agents, real A2A communication between separate processes, hierarchical planning that's inspectable per incident, a human checkpoint that genuinely changes what executes, two independent layers of graceful degradation, and full LangSmith tracing. Thanks for watching."

[ACTION: stop recording]

Timing budget (adjust as you rehearse)

Section	Target	Cumulative
Cold open	0:30	0:30
Architecture	2:15	2:45
Launch system	0:30	3:15
Demo #1 (full run + HITL modify)	3:45	7:00
Edge case (kill agent process)	1:15	8:15
LangSmith / n8n	1:00	9:15
Wrap-up	0:45	10:00

If you're running long: cut the n8n portion first (LangSmith alone still satisfies observability) and tighten the architecture section to just the sequence diagram + the coordination-model tradeoff — those are the two highest-value talking points for the rubric.

If you're running short / want more depth: show the saved `runs/<incident_id>.json` record for the modified-goal incident, or open `incident_response/schemas.py` briefly to show the Pydantic contracts agents hand off between each other.